Student name:_____ ID: _____

CSE 2046/2246 Analysis of Algorithms

Spring 2024 Midterm, duration: 105 minutes

Q1	Q2	Q3	Q4	Q5	SUM
/16	/24	/20	/25	/20	/100

Q-1. (6+5+5=16 pts) Consider the following program. InsertionSort(A), MergeSort(A) and QuickSort(A) functions perform Insertion Sort, Merge Sort and Quick Sort algorithms respectively and return a sorted array. Length of array A can be even or odd with equal probability. In Quick Sort, pivot is selected as the first element of the array.

```
void func1(int A[0..n-1]) {
if (n % 2 = 0) {
      InsertionSort(A); QuickSort(A); }
else {
      MergeSort(A); InsertionSort(A); }
}
```

Sorting a list twice could be meaningless, but this is an exam question. Answer the following:

(i) Indicate the time complexity of func1 for an already sorted input. Describe your reasoning.

if n is even, $\Theta(n)+\Theta(n^2)=\Theta(n^2)$ if n is odd, $\Theta(n\log n)+\Theta(n)=\Theta(n\log n)$ So it is $O(n^2)$.

(ii) Indicate the worst case time complexity of func1 in $\Theta(\cdot)$ form. Describe your reasoning.

In the worst case, n is even & insertion sort takes $\Theta(n^2)$ time. So it is $\Theta(n^2)$ //

(iii) Can we describe the overall time complexity of func1 in $\Theta(\cdot)$ form? Or should we use $O(\cdot)$ notation? Why?

We cannot use $\Theta(\cdot)$ notation, because worst case & best case are not the same. We have to use $O(n^2)$ //

Q-2. (24 pts) Consider the following function.

```
int func2(int A[0..n-1]) {

if (n <= 1)
      return 1;
else {
      if (n%2 == 0) return func2(A[0..n-2])*A[n-1];
      else return func2(A[0..n-3])*func2(A[0..n-2])*A[n-1];
}
```

(a – 8 pts) Give the recurrence relation for <u>number of multiplications</u>, for both even and odd values of n. Don't forget the base case.

$M(n)=M(n-1)+1$ if n is even

$M(n)=M(n-1)+M(n-2)+2$ if n is odd.

$M(0)=\emptyset$

$M(1)=\emptyset$

(b – 16) Solve the recurrence relation using backward substitution. Find the solution for odd values of n. Please be careful: Recurrence relation for even and odd values of n are different. Consider them as two different functions. Don't forget that if n is odd, n-1 will be even!

if n is odd, $M(n)=M(n-1)+M(n-2)+2=M(n-2)+1+M(n-2)+2$

($M(n-1)$: even, $M(n-2)$: odd)

$M(n)=2M(n-2)+3$

$=2(2M(n-4)+3)+3=2^2M(n-4)+3.2+3$

$=2^2(2M(n-6)+3)+3.2+3=2^3M(n-6)+3.2^2+3.2+3$

$=2^{\frac{n-1}{2}}M(n-(n-1))+3.2^{\frac{n-1}{2}-1}+3.2^{\frac{n-1}{2}-2}+\ldots+3.2+3=3.(2^{\frac{n-1}{2}}-1)$

$M(1)=0$

$3(1+2+2^2+\ldots+2^{\frac{n-1}{2}-1})$

Student name:_____ ID: _____

Q-3. (20 pts) Suppose that we have k polynomials $P_1(x), P_2(x) \dots P_k(x)$, all of them have a degree of n and all their coefficients are integers. Polynomials are stored in a two-dimensional integer array A of size $k \times (n+1)$, such that i'th row of array A stores all coefficients of $P_i(x)$.
We have another unsorted array B which stores m integer values.
Given the value of x as an input, our goal is to find the number of polynomials in A with a value that exists in B.

For example, if $A = \begin{bmatrix} 2 & 0 & 3 \\ 1 & 1 & 2 \\ 3 & -1 & -2 \end{bmatrix}$, it means that we have the following 3 polynomials all with degree of 2:
$P_1(x) = 2x^2 + 3$; $P_2(x) = x^2 + x + 2$; $P_3(x) = 3x^2 - x - 2$
And let's say that $B = \{4, 2, 6, 3, 7, 5\}$;
The answer for $x = 1$ is 2 since $P_1(1) = 5 \in B$; $P_2(1) = 4 \in B$; $P_3(1) = 0 \notin B$, i.e. the first two polynomials' values are in B, but the third one's not.

Design an efficient presorting-based algorithm for this problem and give its time complexity. *Notes: Give step-by-step description. Do not give any pseudocode. You don't need to give the details of the algorithm for polynomial evaluation (you can use an already implemented function* `polyeval(Poly[0..n-1],x)` *which returns the value of a polynomial for x), but you have to describe its complexity. Overall time complexity of your algorithm should be given in terms of k, n and m.*

Step 1: Sort Array B using MergeSort. Set counter to zero.
Step 2: For all the polynomials in A repeat Step 3 and Step 4:
Step 3: Evaluate the value of the polynomial using polyeval( )
Step 4: Search the value in sorted B using Binary Search.
If found, increment the counter by 1.
Step 5: Return the counter.

14 pts

Time complexity:
Step 1: $\Theta(m \log m)$
Step 2-3-4: $k(\Theta(n) + O(\log m)) = \Theta(kn) + O(k \log m)$
Overall = $\Theta(m \log m) + \Theta(kn) + O(k \log m)$

6 pts

Student name:_____ ID: _____

Q-4. (25 pts) Given an unsorted array A of negative and positive integers, we are going to find a maximum product subarray, i.e. a subarray with largest product. It is ok to return the product, rather than the array indices.
Here are some examples:
Input: A = [2, 5, -2, 6]
Output: 10
Explanation: [2, 5] has the largest product.

Input: A = [-3, 0 , -1]
Output: 0
Explanation: [-3, -1] is not a subarray. [0] has the largest product.

Input: A = [-4, 2, -5, -6, 0, 7]
Output: 60
Explanation: [2, -5, -6] has the largest product.

(a – 18 pts) Describe a divide-and-conquer algorithm to find the maximum product subarray. Give step-by-step description rather than a pseudocode.

Step 1: If the array has a single element, return it.

Step 2: Divide the array into two parts from the mid element.

Step 3: Find the maximum subarray product in the right part: $MaxP_R$

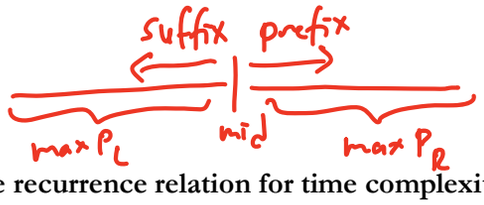
Step 4: Find the maximum subarray product in the left part: $MaxP_L$

Step 5: Find the suffix of left part with maximum product: l_{max}
Find the prefix of right part with maximum product: r_{max}

Step 6: Find the suffix of left part with minimum product: l_{min}
Find the prefix of right part with minimum product: r_{min}

Step 7: return $(\max\{maxP_R, maxP_L, r_{max} \cdot l_{max}, r_{min} \cdot l_{min}\})$

This could be max when both values are negative.



(b – 7 pts) Give the recurrence relation for time complexity of your algorithm. Solve it with Master theorem.

$T(n) = 2T\left(\frac{n}{2}\right) + f(n)$ where $f(n) \in \Theta(n)$

finding the suffix or prefix with max or min product requires one way scan.

$a = 2 \quad b = 2 \quad d = 1$

$a = b^d$, so $T(n) \in \Theta(n \log n)$

If $a < b^d$,	$T(n) \in \Theta(n^d)$
If $a = b^d$,	$T(n) \in \Theta(n^d \log n)$
If $a > b^d$,	$T(n) \in \Theta(n^{\log_b a})$

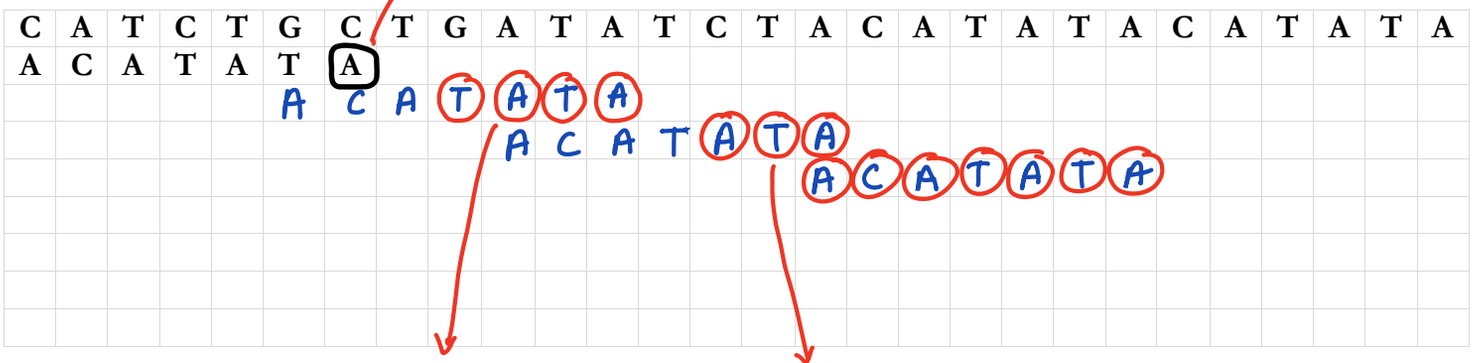
Student name:_____ ID: _____

Q-5 (20 pts) Consider the following DNA string and the pattern.

DNA string: C A T C T G C T G A T A T C T A C A T A T A C A T A T A

Pattern: A C A TA T A

(a – 15 pts) Apply Boyer Moore Algorithm to find an occurrence of the pattern in the given DNA string. Circle the characters in the pattern that get compared with the text. (Show bad-symbol and good-suffix tables. Show all calculations.)



Bad symbol:

T: 1
A: 2
C: 5
G: 7

$k=3$
$d_1 = t('G') - 3 = 4$
$d_2 = 2$
$\max\{d_1, d_2\} = 4$
shift by 4

$k=2$
$d_1 = t('C') - 2 = 3$
$d_2 = 6$
$\max\{d_1, d_1\} = 6$
shift by 6.

Good suffix:

		d_2
$k=1$	A C A T A T A	4
$k=2$	A C A T A T A	6
$k=3$	A C A T A T A	2
$k=4$	A C A T A T A	6
$k=5$	A C A T A T A	6
$k=6$	A C A T A T A	6

(b – 5 pts) How would you modify (or extend) the Boyer Moore algorithm we have described in the class to find ALL occurrences of the pattern, rather than only the first occurrence?

In the good suffix table, we would add an entry for $k=7$ (full match) and d_2 would be 6 for this pattern. Then after a full match, I will shift according to this value. In the above example, we would shift by 6 and find the next occurrence.